

华东师范大学数据科学与工程学院实验报告

课程名称: 数据库管理系统	年级: 2024	上机实践时间: 2026.04.02
指导教师: 周烜	姓名: 孙育泉	
上机实践名称: 问卷系统 I	学号: 10234900421	

目录

I 实验任务	1
II 使用环境	1
III 实验过程	2
III.1 Vibe Coding 大致过程	2
III.2 数据库设计	3
III.3 API 说明	4
III.3.1 全局说明	4
III.3.2 用户认证 (Auth)	4
III.3.3 问卷管理 (Surveys)	5
III.3.4 答卷收集 (Responses)	6
III.3.5 数据统计 (Statistics)	6
III.4 跳转设计	7
III.5 校验设计	9
III.6 测试结果	11
IV 总结	15

I 实验任务

本项目要求开发一个简化版的在线问卷系统，功能类似于“问卷星”或 Google Forms.

系统应允许注册用户创建问卷，并通过 URL 分发问卷，其他用户可以填写问卷，系统可以统计问卷结果.

本项目重点考察：

- MongoDB 数据库设计能力
- 后端程序设计能力
- 数据结构与业务逻辑建模能力
- 条件逻辑与跳转逻辑的实现能力
- 使用 AI 编程工具的能力与规范

本项目不要求复杂界面，可以使用简单网页.

II 使用环境

- 后端数据库：MongoDB
- 编程语言：Python
- 后端框架：FastAPI

- 前端框架: Vue.js
- AI 工具: Gemini

III 实验过程

III.1 Vibe Coding 大致过程

[1] **Prompt** 请你仔细阅读这个文档的要求, 梳理一个任务清单的大纲.

[1] **Result** 理解文档内容, 给出注意事项和任务清单.

[2] **Prompt** 我想使用 uv+python 作为后端语言, 使用 mongodb 作为数据库, 请你先帮我完成整个项目系统的设计, 给我一份清单

[2] **Result** 给出技术栈推荐 (依赖管理与运行环境、Web 框架、数据库驱动、前端); 给出了 MongoDB 数据库设计方案; 给出第一阶段的开发的任务清单.

[3] **Prompt** 好, 我们先完成第一步, 项目的初始化与环境的搭建

[3] **Result** 进行环境的配置, 例如安装 Python、MongoDB、Node.js 等; 搭建后端框架 (FastAPI) 和前端框架 (Vue.js); 配置数据库连接; 创建项目的基本目录结构.

[4] **Prompt** 接下来, 完成步骤 2, 用户认证模块开发

[4] **Result** 更新配置文件; 编写安全与加密工具; 定义数据模型; 编写认证路由; 编写鉴权依赖项; 将路由注册到主程序

[5] **Prompt** 接下来进行问卷管理模块开发这一步

[5] **Result** 定义问卷与题目的数据模型; 编写问卷管理路由; 注册路由到主程序

[6] **Prompt** 接下来, 进行问卷填写与校验逻辑的功能

[6] **Result** 定义问卷与题目的数据模型; 编写问卷管理路由; 注册路由到主程序

[7] **Prompt** 进入 数据统计模块开发

[7] **Result** 定义统计数据模型; 编写统计逻辑路由; 注册路由并测试.

[8] **Prompt** 接下来进行 前端对接与测试, 使用尽可能简单的框架吧

[8] **Result** 让 FastAPI 支持静态网页服务; 让 FastAPI 支持静态网页服务; 编写问卷填写页面;

[9] **Prompt** 我觉得前端还有如下功能没有体现: 1、用户退出账号 2、用户创建问卷 3、查看问卷的信息统计 4、问卷的题目跳转功能 5、修改密码

[9] **Result** 升级后台主页、填写页面

[10] **Prompt** 有如下问题: 1、没有注册选项 2、创建问卷使用 json 太不人性化了 3、前端太丑了, 我还是希望使用前端框架 3、统计结果也是 json, 我希望是有 dashboard 但是支持 json 导出

[10] **Result** 从静态网页升级到前端框架 (Vue.js); 实现用户注册功能; 实现问卷创建的表单化; 设计并实现统计结果的 dashboard, 同时支持 JSON 导出.

Tip

此时, 由于引入了 Vue.js 前端框架改变了项目的文件结构, 原本的项目结构是 `Workspace/survey_backend/`, 现在由于加入了前端, 因此整合为 `Workspace/survey/`, 其中包含 `survey-backend/` 和 `survey-frontend/` 两个子目录, 分别用于后端和前端的开发. 但是, 同时需要修改 `git` 的配置和 `uv` 的虚拟环境.

[11] Prompt 先进行 前端基础设施搭建

[11] Result 初始化项目与安装依赖; 梳理目录结构; 封装 Axios (请求拦截与 Token 管理); 配置前端路由; 全局挂载

[12] Prompt 我使用了 v20 的 node, 接下来进入第二步 认证与基础页面

[12] Result 开发登录与注册页面; 开发后台基础布局

[13] Prompt 但是我现在前端总是说登录失败

[13] Result 在后端放行跨域请求

[14] Prompt 但是我看不见我自己创建的问卷

[14] Result 完善 Dashboard.vue (完整拉取与展示数据)

[15] Prompt 接下来进入问卷动态渲染与跳转逻辑解析

[15] Result 开发问卷填写页面

[16] Prompt 第三步中题目跳转功能在前端还没有实现

[16] Result 修改 Template (增加跳转配置 UI); 修改 Script (增加操作逻辑)

[17] Prompt 接下来完成 数据看板与导出

[17] Result 开发统计看板组件; 注册新路由; 在 Dashboard 绑定跳转入口

[18] Prompt 还有一些细节: 1、用户要求可以设置问卷截至时间 2、文本填空要求可以限制最大最小字数, 在前端没有体现 3、在因为跳转而忽略的题目是必选题时, 无法提交文件, 因为没有做跳过的必选题 4、这些按钮的排列不美观 5、创建问卷的界面我希望也有返回列表的按钮 6、关于跳转还有一个小 bug, 就是多选题的跳转的等于条件只能选择一个选项

[18] Result 解决文本填空字数限制配置 (前端); 解决设置截止时间功能 (前端); 加入处理弹窗的逻辑; 解决跳转逻辑导致必答题被拦截的 Bug (后端); 修复 Dashboard 按钮排列不美观; 在问卷创建界面增加“返回列表”按钮; 修改前端创建器, 修改前端填写引擎, 修改后端硬核校验

[19] Prompt 还有一些问题: 1、我设置了问卷的截止时间之后, 前端没有地方能够体现 2、设置匿名填写的前端部分显示的很奇怪, 不够直观

[19] Result 解决“匿名填写”选项不够直观的问题; 在 Dashboard 中展示截止时间; 在填写页面展示截止时间, 并拦截过期提交

III.2 数据库设计

考虑到问卷系统的功能需求, 我们设计了以下 3 个集合:

(1) users 集合: 用于处理系统的注册、登录和鉴权.

- _id: ObjectId, 主键
- username: String(唯一索引)
- password_hash: String, 密码哈希值
- created_at: DateTime (注册时间)

(2) surveys 集合: 将“题目”作为数组嵌套在“问卷(Survey)”文档中, 这是因为问卷和它的题目是强聚合关系, 通常是一起读取的. 嵌套设计可以避免关系型数据库中频繁的 JOIN 操作, 充分发挥 MongoDB 的文档模型优势.

- _id: ObjectId, 主键
- title: String, 问卷标题
- description: String, 问卷描述
- creator_id: ObjectId, 关联到 users 集合的用户 ID
- is_anonymous: Boolean, 是否允许匿名填写

- `deadline`: `DateTime`, 问卷截止时间
 - `questions`: Array of Objects 对象, 题目列表
 - `q_id`: String, 题目 ID (唯一标识符)
 - `type`: String (“single”, “multiple”, “text”, “number”)
 - `title`: String
 - `is_required`: Boolean
 - `options`: Array of Strings (仅选择题)
 - `constraints`: Object (针对特定题型的限制条件)
 - 多选题限制: `min_select`, `max_select`, `exact_select`
 - 文本题限制: `min_length`, `max_length`
 - 数值题限制: `min_value`, `max_value`, `is_integer`
 - `jump_logic`: Array of Objects (跳转逻辑配置)
 - `condition_value`: Any (触发跳转的选项值或者填空值)
 - `target_q_id`: String (跳转目标题目 ID)
- (3) `responses` 集合: 答卷数据, 必须与问卷分离. 随着填写人数增加, 答卷数据会快速膨胀, 如果嵌套在 `surveys` 中会导致单个文档超出 MongoDB 的 16MB 限制.
- `_id`: `ObjectId`
 - `survey_id`: `ObjectId`, 关联到 `surveys` 集合的问卷 ID
 - `user_id`: `ObjectId` (如果未匿名且已登录, 记录填写者)
 - `submitted_at`: `DateTime`
 - `answers`: Array of Objects (用户的具体回答)
 - `q_id`: String, 题目 ID
 - `value`: Any (单选是字符串, 多选是数组, 填空是文本或数字)

III.3 API 说明

III.3.1 全局说明

- 基础路径 (Base URL): `http://localhost:8000`
- 鉴权方式: 采用 OAuth2 规范的 JWT (JSON Web Token) 机制. 对于受保护的接口, 需在 HTTP 请求头中携带 `Authorization: Bearer <token>`.

III.3.2 用户认证 (Auth)

(1) 用户注册

- 接口地址: `POST /api/auth/register`
- 功能描述: 创建新用户账号
- 访问权限: 公开
- 请求格式 (`application/json`):

```
1 {
2   "username": "testuser",
3   "password": "password123"
4 }
```

json

- 响应结果: 成功返回 200 OK 及成功信息; 失败返回 400 Bad Request (如用户名已存在或格式不符).

(2) 用户登录

- 接口地址: `POST /api/auth/login`

- 功能描述：用户登录并获取 JWT 令牌
- 访问权限：公开
- 请求格式 (application/x-www-form-urlencoded)：注意遵循 OAuth2 规范，需要使用 FormData 传递。
 - username: 用户名
 - password: 密码
- 响应结果(application/json):

json

```
1 {
2   "access_token": "eyJhbGciOiJIUz... ",
3   "token_type": "bearer"
4 }
```

III.3.3 问卷管理 (Surveys)

(1) 创建问卷

- 接口地址：POST /api/surveys
- 功能描述：提交包含问卷基础信息、题目、逻辑跳转及约束条件的结构化数据。
- 访问权限：需登录
- 请求格式 (application/json):

json

```
1 {
2   "title": "问卷标题",
3   "description": "问卷描述/感谢语",
4   "is_anonymous": true,
5   "questions": [
6     {
7       "q_id": "q_abc123",
8       "type": "single | multiple | text | number",
9       "title": "题目内容",
10      "is_required": true,
11      "options": ["选项1", "选项2"],
12      "constraints": {
13        "min_select": 1, "max_select": 2, // 针对多选题
14        "min_value": 0, "max_value": 100, // 针对数字题
15        "min_length": 0, "max_length": 500 // 针对文本题
16      },
17      "jump_logic": [
18        { "condition_value": "选项1", "target_q_id": "q_xyz789" }
19      ]
20    }
21  ]
22 }
```

- 响应结果：返回创建成功的问卷详情及分配的 id.

(2) 获取当前用户的问卷列表

- 接口地址：GET /api/surveys
- 功能描述：拉取当前登录用户创建的所有问卷，用于在 Dashboard 中展示。
- 访问权限：需登录
- 响应结果(application/json)：返回问卷数组对象，包含问卷标题、状态、创建及截止时间等。

(3) 更新问卷状态 (发布/关闭/设置截止时间)

- 接口地址: PUT /api/surveys/{survey_id}/status
- 功能描述: 修改问卷的发布状态或设定失效时间.
- 访问权限: 需登录 (仅限创建者)
- 请求格式 (application/json):

json

```
1 {
2   "is_active": true,
3   "deadline": "2026-04-10T12:00:00Z" // 可选
4 }
```

- 响应结果: 返回更新后的问卷基础信息.

(4) 获取问卷详情 (用于渲染填写页)

- 接口地址: GET /api/surveys/{survey_id}
- 功能描述: 获取整份问卷的题目配置和跳转逻辑.
- 访问权限: 公开
- 响应结果(application/json): 返回完整的问卷树状结构数据.

III.3.4 答卷收集 (Responses)

(1) 提交答卷

- 接口地址: POST /api/surveys/{survey_id}/responses
- 功能描述: 接收用户提交的答案, 执行后端的“硬核校验”(包括必答、数字范围、多选数量及跳题免责逻辑).
- 访问权限: 视问卷设定的 is_anonymous 字段而定 (公开或需登录).
- 请求格式 (application/json):

json

```
1 {
2   "answers": [
3     { "q_id": "q_abc123", "value": "选项1" },
4     { "q_id": "q_def456", "value": ["选项1", "选项2"] },
5     { "q_id": "q_ghi789", "value": 25 },
6     { "q_id": "q_jkl012", "value": "这是一段文本回答" }
7   ]
8 }
```

- 响应结果:
 - ▶ 200 OK: 提交成功, 返回落库后的答卷文档 ID.
 - ▶ 400 Bad Request: 触发格式、必答或约束校验失败, 返回具体错误 detail.
 - ▶ 403 Forbidden: 问卷未发布或已过截止时间.

III.3.5 数据统计 (Statistics)

(1) 获取问卷统计报表

- 接口地址: GET /api/surveys/{survey_id}/statistics
- 功能描述: 聚合计算该问卷的所有答卷数据, 用于在前端渲染图表看板.
- 访问权限: 需登录 (严格限制为该问卷的创建者才可访问).
- 响应结果(application/json):

json

```
1 {
2   "survey_id": "60d5ec ... ",
3   "total_submissions": 42,
4   "questions": [
5     {
```

```

6      "q_id": "q_abc123",
7      "type": "single",
8      "title": "题目内容",
9      "total_responses": 40,
10     "option_counts": { "选项1": 30, "选项2": 10 } // 选择题特有
11   },
12   {
13     "q_id": "q_ghi789",
14     "type": "number",
15     "title": "年龄分布",
16     "total_responses": 42,
17     "average": 22.5 // 数字题特有
18   },
19   {
20     "q_id": "q_jkl012",
21     "type": "text",
22     "title": "您的建议",
23     "total_responses": 5,
24     "text_answers": ["界面好看", "希望能增加XX功能"] // 文本题特有
25   }
26 ]
27 }

```

III.4 跳转设计

在传统的简单表单中，题目是线性平铺的；但加入了跳转逻辑后，整份问卷就变成了一个有向无环图。

首先，在数据结构设计上，每一个题目都有一个 `jump_logic` 数组。我们没有采用“跳过某题”的设计，而是采用“直接跳转到目标题”。因为跳转到目标题意味着“从当前题到目标题中间的所有题目，统统被屏蔽”。这在算法实现上更加清晰。

json

```

1 {
2   "q_id": "q_1",
3   "type": "single",
4   "title": "您是否使用过Linux? ",
5   "options": ["是", "否"],
6   "jump_logic": [
7     {
8       "condition_value": "否", // 触发条件
9       "target_q_id": "q_5"    // 命中了跳去哪
10    }
11  ]
12 }

```

前端的核心任务是：实时监听输入 → 评估图表连线 → 隐藏中间节点。

这部分逻辑集中在 `SurveyFill.vue` 的 `evaluateJumpLogic` 函数中。

在 `Vue` 模板中，我们给所有的输入组件（单选、多选、数字输入）都绑定了 `@change="evaluateJumpLogic"`。这意味着只要用户改了答案，整个状态机就会重新运转一次。

js

```

1 // 被隐藏的题目 ID 黑名单
2 const hiddenQuestions = ref(new Set())
3
4 const evaluateJumpLogic = () => {

```



```

5  const newHidden = new Set() // 每次重新计算，避免状态残留
6  const questions = survey.value.questions
7
8  // 1. 从上到下顺序遍历所有题目
9  for (let i = 0; i < questions.length; i++) {
10   const q = questions[i]
11   if (!q.jump_logic || q.jump_logic.length === 0) continue
12
13   const currentValue = formData[q.q_id] // 获取用户当前选的值
14   let targetId = null
15
16   // 2. 检查当前答案是否命中了规则
17   for (const logic of q.jump_logic) {
18     if (q.type === 'multiple' && Array.isArray(logic.condition_value)) {
19       // 【亮点】：多选题的集合论！
20       // 要求设定的规则组合（condition_value）必须是用户选择组合（currentValue）的子集
21       const isMatch = logic.condition_value.length > 0 &&
22         logic.condition_value.every(v =>
23           currentValue.includes(v))
24       if (isMatch) { targetId = logic.target_q_id; break }
25     }
26     // ... （省略单选和数字的简单等值判断）
27   }
28
29   // 3. 执行“图节点屏蔽”操作
30   if (targetId) {
31     // 找到目标题目在数组中的位置
32     const targetIndex = questions.findIndex(item => item.q_id === targetId)
33     // 把当前题（i）和目标题（targetIndex）之间的所有题目 ID 扫进黑名单
34     if (targetIndex > i) {
35       for (let j = i + 1; j < targetIndex; j++) {
36         newHidden.add(questions[j].q_id)
37       }
38     }
39   }
40   // 4. 更新响应式状态，触发视图重绘
41   hiddenQuestions.value = newHidden
42 }

```

在前端，我们利用 Vue 的 `v-show="!hiddenQuestions.has(q.q_id)"`，瞬间将黑名单里的题目在屏幕上隐藏。然后，在提交进行校验的时候，使用 `if (hiddenQuestions.value.has(q.q_id)) continue` 保证被隐藏的题目答案绝不发给后端。

最后，在后端也要写一遍跳题逻辑，这是因为如果在前端跳过了一个必答题，而后端不知道，进行检测的时候会发现有必答题没有作答，就会报错返回 400。这体现在 `responses.py` 的 `submit_response` 接口中：

py

```

1 # 后端的黑名单集合
2 hidden_q_ids = set()
3
4 # 同样必须严格按顺序遍历题目（因为跳转是有方向的）
5 for i, q_def in enumerate(questions):
6     q_id = q_def["q_id"]
7

```



```

8      # 1. 免死金牌：如果你在黑名单里，跳过所有非空、格式、字数等强制校验
9      if q_id in hidden_q_ids:
10         continue
11
12     # ... （执行正常的必答、边界值硬核校验） ...
13
14     # 2. 校验通过后，后端同样评估这道题会不会触发未来的跳题
15     jump_logic = q_def.get("jump_logic", [])
16     target_id = None
17     for logic in jump_logic:
18         cond_val = logic.get("condition_value")
19         t_id = logic.get("target_q_id")
20
21         # 【亮点】：后端的集合子集判断，逻辑与前端 Vue 代码遥相呼应
22         if q_type == "multiple" and isinstance(value, list) and
23            isinstance(cond_val, list):
24             if len(cond_val) > 0 and set(cond_val).issubset(set(value)):
25                 target_id = t_id; break
26
27     # 3. 如果命中，把中间的题目押入黑名单
28     if target_id:
29         # ... 找到 target_idx ...
30         for j in range(i + 1, target_idx):
31             hidden_q_ids.add(questions[j]["q_id"])

```

III.5 校验设计

因为前端的代码运行在用户的浏览器中，极易容易被篡改或者绕过。因此，我们在设计校验机制时，采用了经典的“纵深防御”与“双重防线”架构。

首先，第一道防线：在处理任何一道具体题目的答案之前，系统必须先从“宏观层面”判断这份答卷是否具有合法身份。这部分逻辑集中在后端 `app/routers/responses.py` 的入口处。

py

```

1 # 1. 存在性与发布状态校验
2 if not survey.get("is_active", False):
3     raise HTTPException(status_code=403, detail="该问卷尚未发布或已关闭")
4
5 # 2. 截止时间校验（时区安全的 UTC 比较）
6 deadline = survey.get("deadline")
7 if deadline and datetime.now(timezone.utc) >
8    deadline.replace(tzinfo=timezone.utc):
9     raise HTTPException(status_code=403, detail="该问卷已超过截止时间，停止收集")
10
11 # 3. 权限校验（匿名控制）
12 if not survey.get("is_anonymous", False) and current_user is None:
13     raise HTTPException(status_code=401, detail="该问卷不允许匿名填写，请先登录")

```

- 短路求值：把这层校验放在最前面，一旦命中立刻抛出 403 或 401 错误，拒绝执行后续消耗性能的逐题比对，保护服务器资源。
- 严格的时区对齐：在处理截止时间时，统一转换为 `timezone.utc` 进行比较，避免了因为服务器物理所在时区和用户时区不同导致的时间差问题。

第二道防线：前端的校验防线，其核心目的不是为了安全，而是为了用户体验。它要在用户点击“提交”的一瞬间，立刻指出哪里漏填了，而不是等网络请求发出去、后端报错后再把错误塞回给用户。

js

```

1 for (const q of survey.value.questions) {
2   // 豁免跳过的题目
3   if (hiddenQuestions.value.has(q.q_id)) continue
4
5   const val = formData[q.q_id]
6
7   // 简单的必填项拦截（体验层防线）
8   if (q.is_required) {
9     if (val === null || val === '' || (Array.isArray(val) && val.length === 0))
10    {
11      ElMessage.warning(`第 ${survey.value.questions.indexOf(q) + 1} 题是必填项，
12        请填写后再提交！`)
13      return // 阻断网络请求
14    }
15  }
16 }

```

配合 Element Plus 的 v-loading 状态和及时的 ElMessage 气泡提示，前端在第一秒钟就帮用户规避了最容易犯的“漏题”错误。

最后，第三道防线：后端不仅要防漏填，还要防“脏数据”（比如要求填数字结果传了字母，要求最多选 2 项结果传了 3 项，或者传了一个根本不存在的选项）。

由于问卷是由 JSON 动态生成的，后端不能像传统的静态模型（如固定的 User 模型）那样写死校验规则。后端必须充当一个“解释器”，读取问卷的 constraints 字典，并动态执行校验。

py

```

1 # 1. 单选/多选：脏选项拦截（防止恶意注入）
2 if q_type == "multiple":
3     # 确保用户传来的选项，必须是在数据库里预设好的选项的子集
4     if not all(v in q_def.get("options", []) for v in value):
5         raise HTTPException(status_code=400, detail=f"题目 '{q_def['title']}' 存在不合法选项")
6
7     # 动态执行数量约束
8     select_count = len(value)
9     if "min_select" in constraints and select_count < constraints["min_select"]:
10        raise HTTPException(status_code=400, detail="...")
11
12 # 2. 数字填空：类型与边界安全检查
13 elif q_type == "number":
14     try:
15         num_value = float(value) # 强转测试，防注入非数字字符串
16     except (ValueError, TypeError):
17         raise HTTPException(status_code=400, detail=f"题目 '{q_def['title']}' 必须是有效数字")
18
19     if constraints.get("is_integer", False) and not num_value.is_integer():
20         raise HTTPException(...) # 整数约束
21
22     if "max_value" in constraints and num_value > constraints["max_value"]:
23         raise HTTPException(...) # 上界约束

```

- 白名单机制 (防脏数据): 对于选择题, 后端不信任前端传来的任何字符串, 而是用 `all(v in q_def.get("options", []) for v in value)` 进行白名单核对. 就算黑客用 Postman 提交了 `"value": "我是来捣乱的选项"`, 也会被立刻击落.
- 动态多态校验: 利用 `if/elif` 结构, 针对 `q_type` 执行完全不同的类型转换和校验逻辑 (如检测 `len()` 还是转换 `float()`), 完美契合了文档中对不同题型施加不同限制的严苛要求.

III.6 测试结果

在前期, 没有前端的时候, 测试后端 API 使用的是 FastAPI 自带的 Swagger UI, 如 图 1 所示:

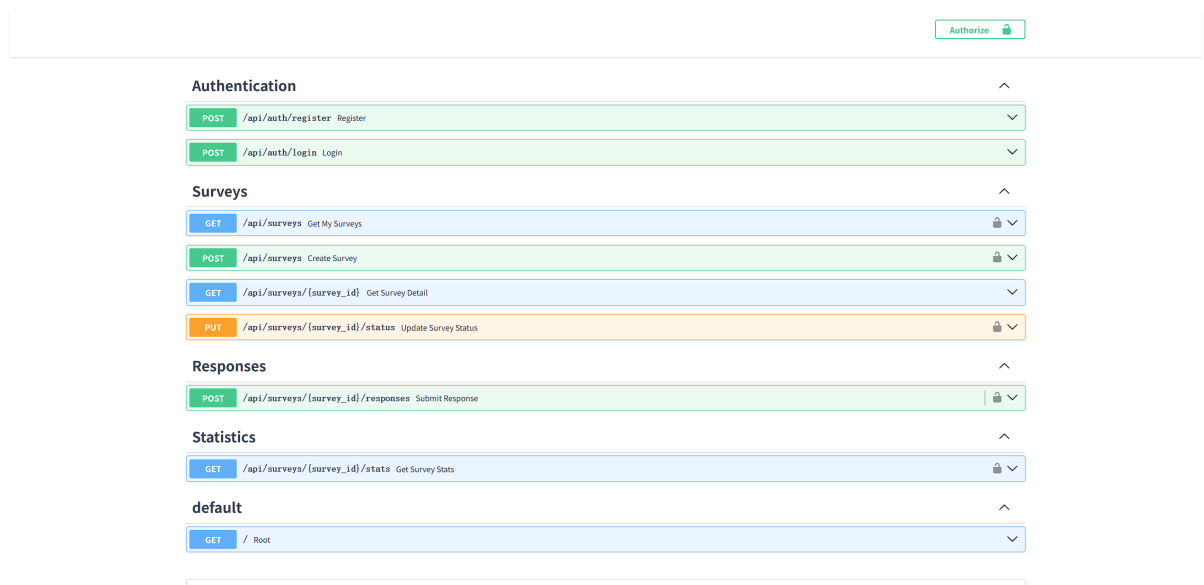


图 1 localhost:8000/docs

此后, 随着前端的加入, 我们主要在 Vue.js 的界面上进行功能测试. 以下是一些关键功能的测试截图.

首先是登录界面.



图 2 登录界面

然后, 登陆成功后进入用户中心 Dashboard, 可以看到自己创建的问卷列表.

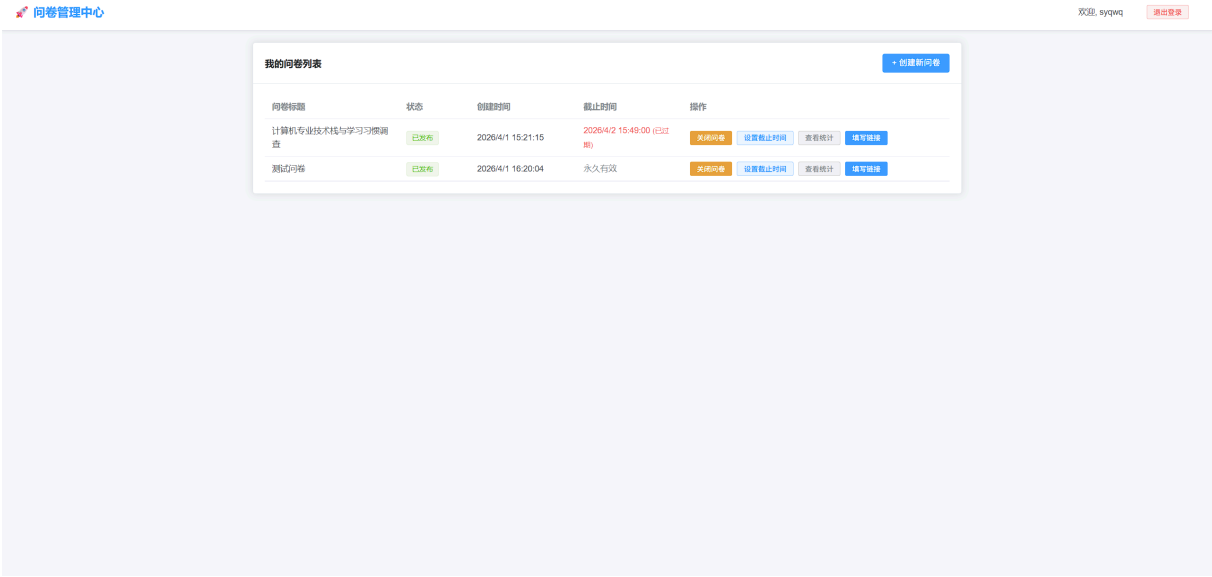


图 3 Dashboard

可以设置问卷的截止时间，截止时间过期后，前端会有明显的红色提示，并且在问卷填写界面有禁止提交的提示，如 图 4 和 图 5 所示.

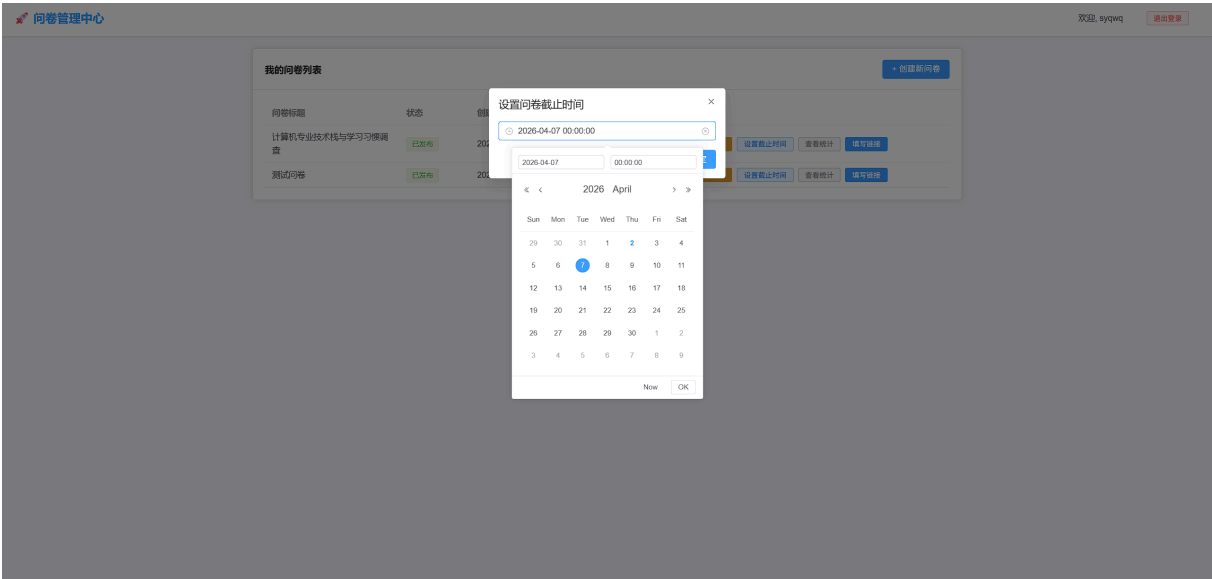


图 4 设置截止时间

图 5 填写界面

在 Dashboard 界面同时可以查看发布的问卷的统计结果, 如 图 6 所示.

图 6 统计结果

图 7 是创建问卷的界面，可以看到题目的配置项非常丰富，支持设置题目类型、选项、必答、约束条件以及跳转逻辑。

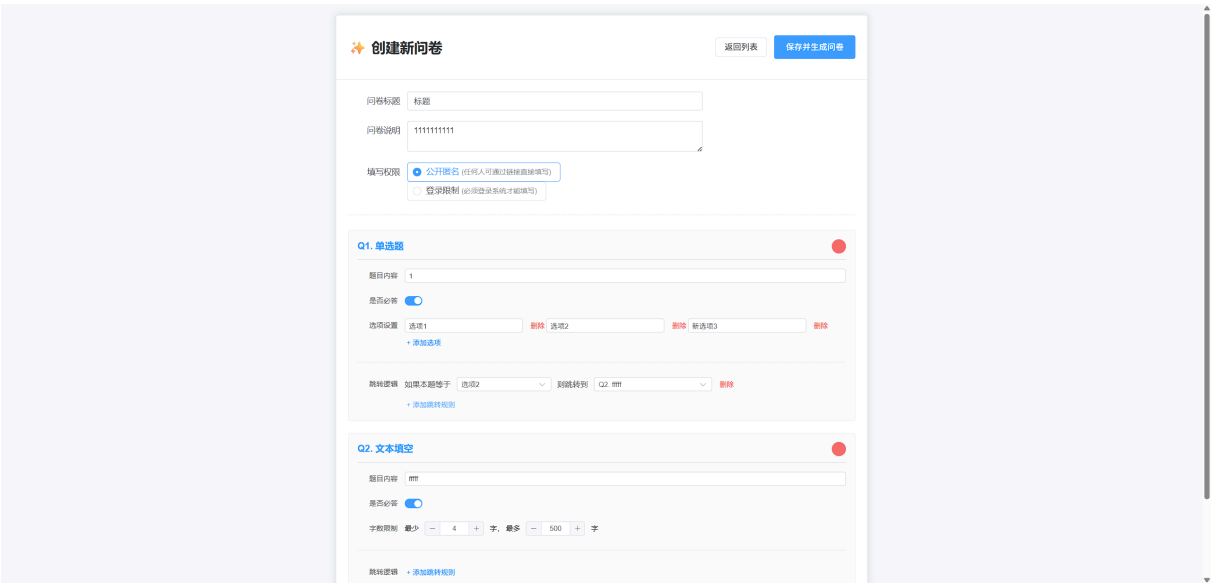


图 7 创建问卷界面

在填写界面，可以看到关于必选题必须作答和题目约束条件没满足无法提交的提示，如 图 8 和 图 9 所示.



图 8 必答题未填提示

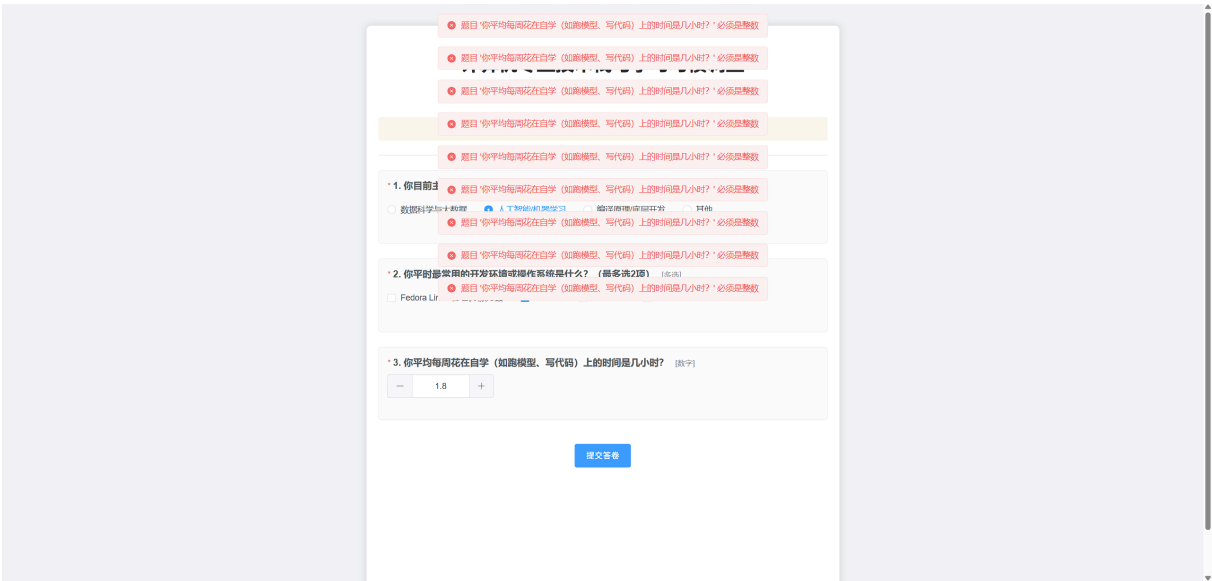


图 9 数字题边界检查提示

IV 总结

本次实验成功开发了一个具备核心功能的简化版在线问卷系统。系统使用了 MongoDB 作为后端数据库，Python 和 FastAPI 作为后端框架，Vue.js 作为前端框架，并结合 Gemini 进行了 AI 辅助开发。充分发挥了 MongoDB 的非关系型文档模型优势，将题目嵌套在问卷文档中以避免频繁的 JOIN 操作。系统实现了用户注册登录、问卷创建、问卷填写、跳转逻辑以及数据统计等功能。特别是在跳转逻辑的设计与实现上，前后端代码形成了完美的呼应。在校验机制上，采用了“纵深防御”与“双重防线”的架构，既保证了系统的安全性，又提升了用户体验。前端的实时校验和后端的严格校验相辅相成，有效防止了漏填和脏数据的提交。总的来说，本次实验不仅完成了功能要求，还在设计和实现上体现了对复杂业务逻辑的深入理解和巧妙处理。

i Info

- 运行命令如下：
 - (1) 启动前端： `cd survey-frontend && npm run dev`
 - (2) 启动后端： `cd survey-backend && uv run uvicorn main:app --reload`
- 同步后端 Python 依赖： `uv sync`
- 完整代码及相关文件见 Github repo: <https://github.com/syqwq-OMG/survey>